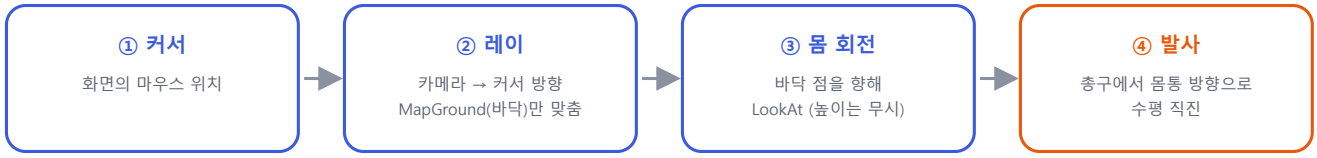
COMBAT · PROJECTILE AIM

커서는 몹 위에 있는데, 왜 빗나갈까

쿼터뷰 카메라는 비스듬히 내려다본다. 그런데 조준은 "바닥"만 본다. 이 둘이 어긋나는 지점을 그림으로 정리했다.

지금의 조준 흐름



CharacterMovement.RotateToMouse() → SpawnProjectiles. 문제는 ②: 레이가 몹을 무시하고 바닥까지 간다.

0.59m

옆에 있는 몹의 **머리**를 찍었을 때 탄이 비껴가는 거리 (필드 카메라)

0.32m

맞으려면 이 안쪽으로 지나가야 함 (몹 반지름 0.265 + 탄 반폭 0.057)

0.64m

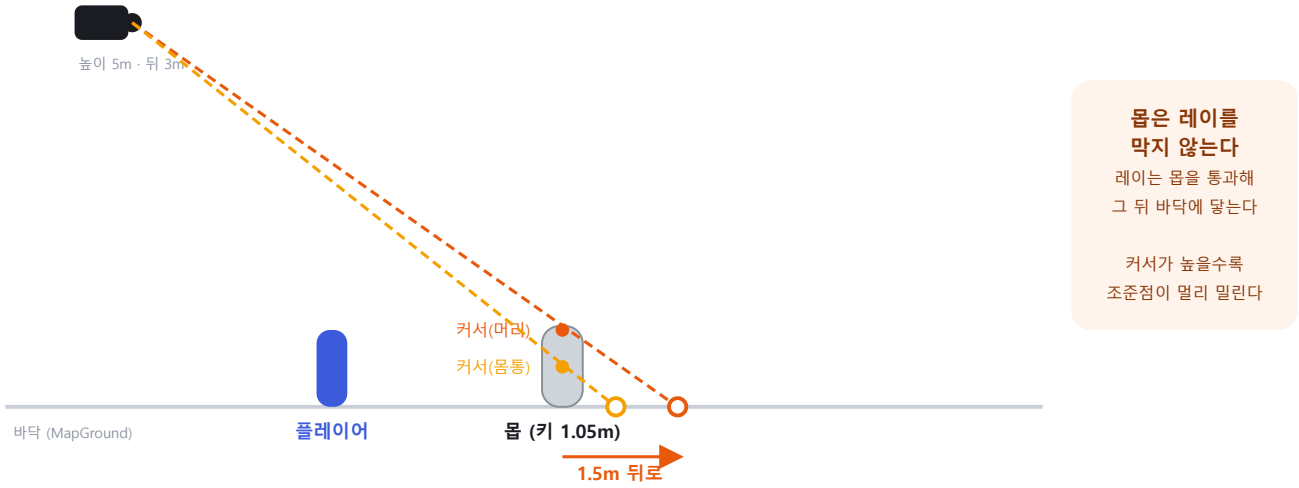
보스 던전 카메라에선 **몸통**만 찍어도 이만큼 비껴감

한 줄 요약

몹 위에 커서를 올리면 레이는 몹을 **통과해 뒤쪽 바닥**에 닿는다. 조준점이 **카메라 반대쪽으로 밀리고**, 몹이 **옆에** 있으면 그 밀림이 쏘는 방향을 틀어서 탄이 옆으로 지나간다. 결국 플레이어는 "몹 **발밑**을 찍어야 맞는다"를 배워야 한다.

1. 옆에서 보면: 조준점이 몹 뒤로 밀린다

필드 카메라 (높이 5m · 뒤 3m · 약 59°), 앞쪽 3m의 몹

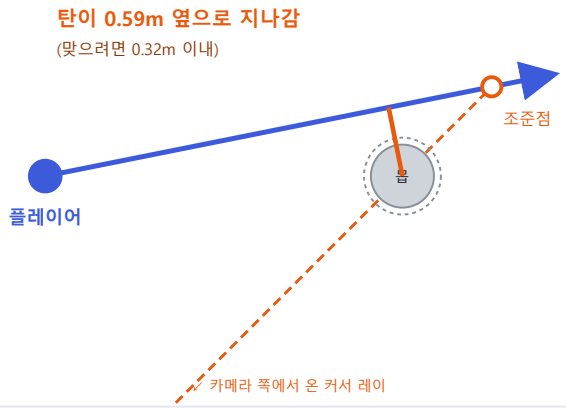


밀리는 거리 = 커서 높이 ÷ (카메라 높이 − 커서 높이) × 카메라에서 몹까지의 수평 거리. 머리(1m)면 1.5m, 몸통(0.52m)이면 0.7m.

2. 위에서 보면: 몹이 어디 있느냐에 따라 갈린다

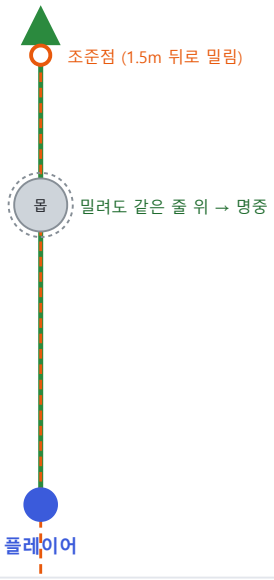
같은 "머리 조준"인데 옆 몹은 빗나가고, 앞 몹은 맞는다

옆에 있는 몹 (오른쪽 3m) · 머리 조준



밀리는 방향 = 카메라 반대쪽 → 쏘는 방향과 어긋남
그래서 몹이 옆에 있으면 탄이 옆으로 빠진다

앞에 있는 몹 (화면 위쪽 3m) · 머리 조준

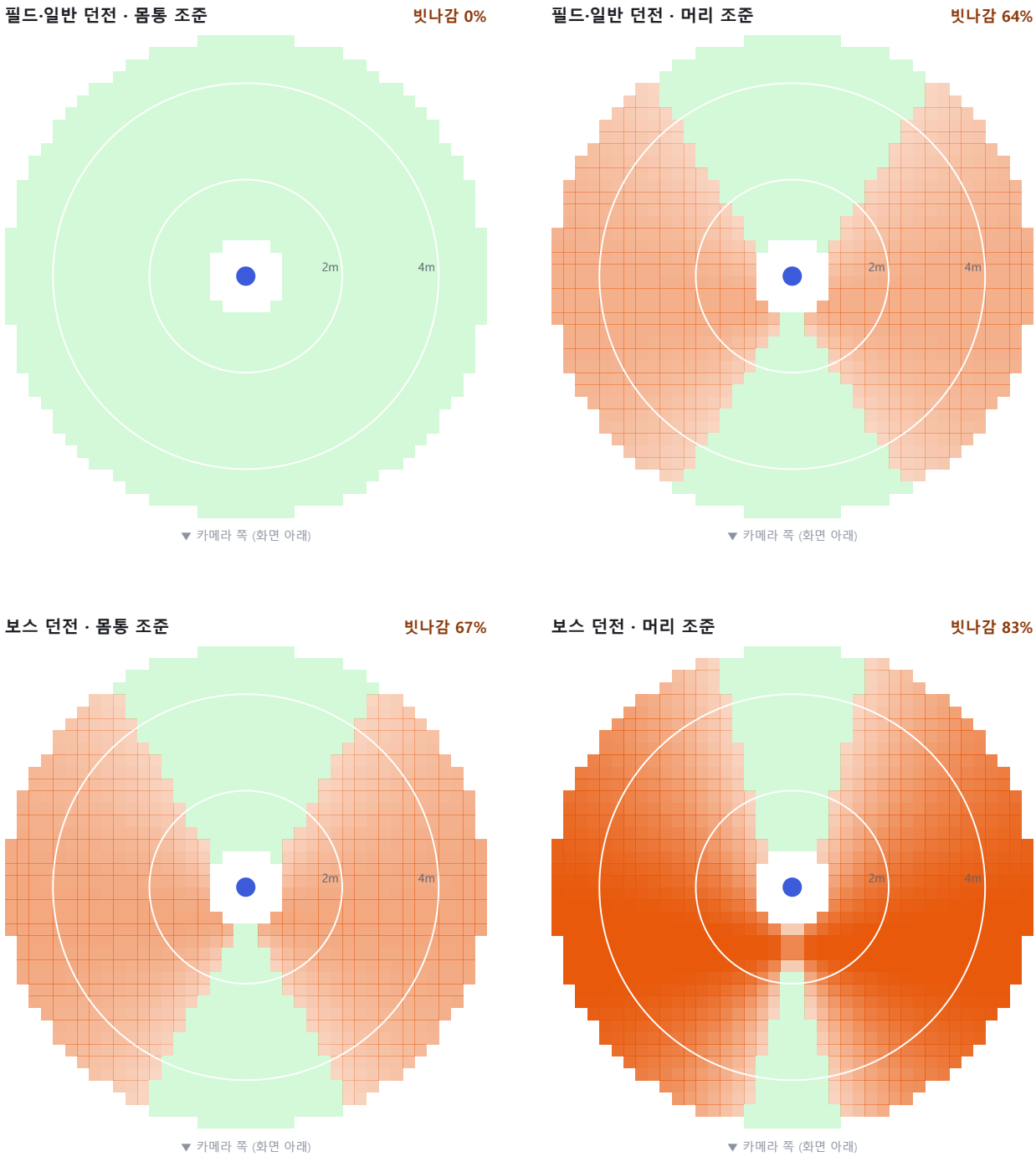


밀리는 방향 = 쏘는 방향과 같음
거리만 멀어지고 방향은 그대로라 맞는다

주황 점선 = 커서 레이를 바닥에 내려 그린 선. 파란/초록 화살표 = 실제 탄 경로. 점선 원 = 탄이 이 안을 지나야 명중. 카메라는 두 그림 모두 화면 아래쪽(패널 밖).

3. 플레이어 주변 어디에 있는 몹이 빗나가나

가운데 파란 점이 플레이어, 아래쪽이 카메라. 칸마다 "그 자리의 몹을 커서로 찍고 쐈을 때"를 계산했다. **초록 = 명중**, **주황 = 빗나감** (진할수록 많이 빗나감). 반경 5m까지.

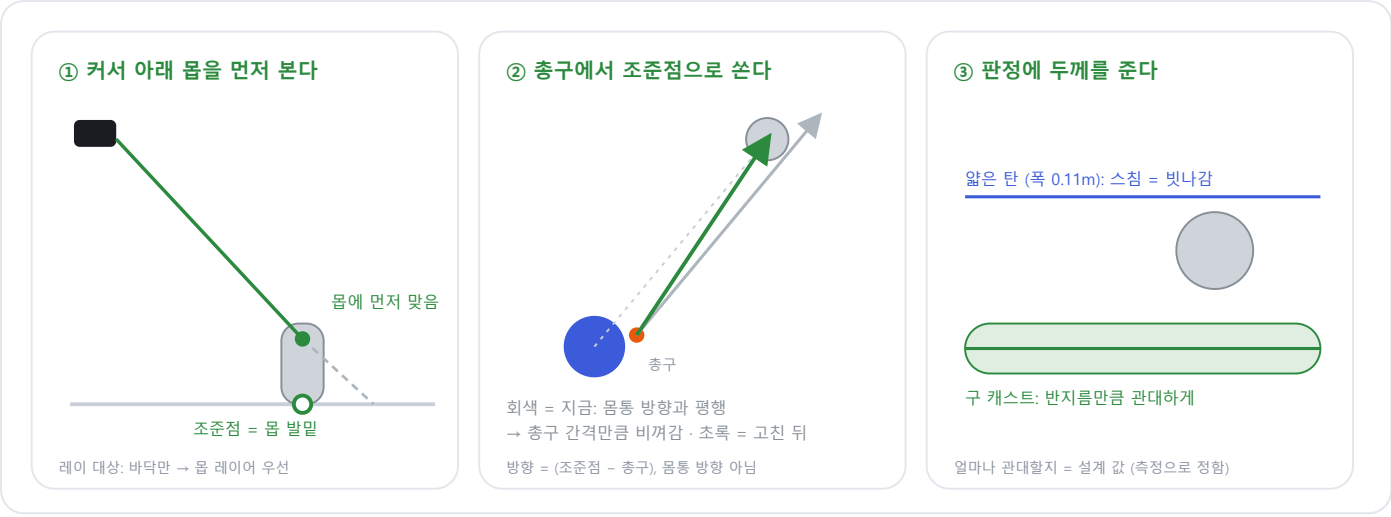


세로(화면 위아래) 줄의 몹은 항상 맞고, 옆으로 갈수록 빗나간다. 거리와는 거의 상관없다 — 옆 몹의 빗나간 거리는 커서 높이와 카메라 위치만으로 정해진다.

읽는 법

- **필드·일반 던전**: 몸통 조준은 옆 몹도 0.31m로 아슬아슬하게 맞는다. **머리**를 찍으면 옆쪽 대부분이 빗나간다.
- **보스 던전**: 카메라가 더 낮고 멀어서(약 39°) 밀림이 커진다. **몸통만 찍어도** 옆쪽이 빗나간다.
- 모든 계산은 **탄이 플레이어 중심에서 나간다**고 가정했다. 실제로는 총구가 옆에 붙어 있어 오차가 더 커지거나 줄어들 수 있다.

4. 고치는 방향 세 가지



우리 쪽 해법	Naughty Dog TLOU2 (GDC 2024)	무엇을 막나
① 커서 아래 몹 우선	"Reticle is King" — 조준점이 적 위면 맞아야 한다	카메라 레이와 바닥 조준점의 어긋남
② 총구 → 조준점	카메라 판정 결과로 캐릭터 탄 궤적을 역산	총구와 몸 중심의 간격
③ 판정 두께	레이캐스트 → 구 캐스트, 반지름을 살짝 키움	스치듯 지나가는 아까운 빗나감

다음: 계산이 맞는지 재보기

- 사격 봇이 몹의 **발 · 몸통 · 머리**에 커서를 올리고 쏜다. 몹 위치는 플레이어 주변 8방향.
- 탄마다 **명중 여부**와 **빗나간 거리**(몹 콜라이더까지 가장 가까웠던 거리)를 기록 → 위 지도의 실측판.
- ①②③을 하나씩 켜고 같은 조건으로 다시 재서 비교한다.

이 문서의 숫자는 계산값

카메라 오프셋(씬 값), AR 몹 캡슐(반지름 0.265m · 키 1.05m), 탄 박스(폭 0.114m)는 프로젝트에서 읽은 값이다. 총구 위치·몸 회전 보간·캐릭터마다 다른 무기는 반영하지 않았다. 실제 체감은 측정으로 확인해야 한다.

생성: docs/Combat/tools/aim_miss_gen.py · 근거 코드: CameraController(_delta) · CharacterMovement.RotateToMouse · SpawnProjectiles · BulletController · 참고: Derek Mattson, "A Second Shot", GDC 2024